

Contents

Mi Turno — Complete Setup Guide	1
For ChatGPT: how to guide a non-technical friend to deploy his own private copy	1
0. READ THIS FIRST (instructions for ChatGPT)	1
1. What this app is (context for you, ChatGPT)	2
2. PHASE 1 — Get the code into his own GitHub	3
3. PHASE 2 — His own Supabase project	3
4. PHASE 3 — Put his keys into the app	4
5. PHASE 4 — Publish it and install it on the iPhone	5
6. PHASE 5 — Notifications (optional, do it last)	7
7. Making the app his (first-run personalization)	8
8. Things ChatGPT must remember while helping	8
9. Troubleshooting (symptom -> cause -> fix)	9
10. Final checklist (go through this with him at the end)	10
11. One-paragraph summary (if he asks “what am I actually doing?”)	11

Mi Turno — Complete Setup Guide

For ChatGPT: how to guide a non-technical friend to deploy his own private copy

0. READ THIS FIRST (instructions for ChatGPT)

You are guiding **a non-technical person** through deploying his own private copy of an existing iPhone web app called **Mi Turno** (a personal habits / identity / workout / meals tracker). He is **not a developer**. He does **not** need to write any code — the app is already finished.

Your job: walk him step by step until the app is installed and fully working on his iPhone, with his own private backend.

Rules for you, ChatGPT

1. **Speak Spanish with him.** He is a Spanish speaker and the app’s interface is in Spanish. This document is in English for your benefit only.
2. **One step at a time.** Never dump the whole guide on him. Give him one action, wait for him to confirm or send a screenshot, then continue.
3. **Ask for screenshots** whenever he is on an unfamiliar screen (Supabase, Netlify, GitHub). Web dashboards change their layout often — verify what he actually sees instead of assuming.
4. **Never ask for secrets.** He must never paste into the chat: his VAPID_PRIVATE key, his Supabase database password, or his service_role key. The Project URL and the publishable/anon key are public by design and are safe to discuss.
5. **Explain the “why” in one sentence** before each phase so he doesn’t feel lost.
6. **Expect the process to take about 45-60 minutes.** Tell him that upfront and suggest doing it on a computer with his iPhone nearby.
7. **If a step fails, do not skip it.** Ask for the exact error message or a screenshot and troubleshoot using Section 9.
8. **Do not modify any code beyond the exact lines listed in Phase 3.** Everything else in the app is already configured and tested.

What he needs before starting

- A computer (Windows or Mac) with a modern browser.

- An iPhone running **iOS 16.4 or newer** (required for notifications).
- Free accounts he will create along the way: **GitHub, Netlify, Supabase**.
- The project files (a .zip folder or a GitHub repository link) given to him by his friend Patricio.

Privacy note (tell him this — it matters)

He will create **his own Supabase project**. That means his own database, completely separate from Patricio's. **Patricio cannot see any of his data** — not his journal, not his goals, not his habits. There is no shared server and no shared account. The app code is shared; the data is not.

1. What this app is (context for you, ChatGPT)

Mi Turno is a Progressive Web App (PWA). Key facts:

- Vanilla **HTML / CSS / JavaScript**. No framework, no build step, no compilation. The files are served exactly as they are.
- It is installed on iPhone by adding it to the Home Screen from Safari. It then runs full screen with its own icon, works offline, and can send notifications.
- **Local-first**: all data is saved on the phone (localStorage + IndexedDB for photos). Supabase is used only for (a) cloud backup / sync between devices, and (b) firing scheduled notifications.
- Features: daily habits, “commitments” (things you’re quitting, with streaks), meals (menu or exchange-portion system), tasks and calendar, workouts with a rest-timer player, identity/goal pages with a vision board, progress charts, local backup, cloud sync, and push notifications.

File structure

index.html	Entry point
css/styles.css	All styles
js/config.js	Seed data + KEYS <- the only file he edits
js/app.js	Core, state, all views
js/gym.js	Routines + workout player
js/reorder.js	Drag-to-reorder + app init
js/sync.js	Supabase auth/sync + push notifications
sw.js	Service worker (offline + push)
manifest.webmanifest	PWA manifest
icons/	App icons
generar-llaves.html	Notification key generator (open in browser)
supabase/functions/send-reminders/index.ts	Server function for notifications

The five phases

Phase	What happens	Roughly
1	Get the code into his own GitHub repository	10 min
2	Create his own Supabase project (database + login)	10 min
3	Put his own keys into the app	5 min
4	Publish on Netlify + install on iPhone	10 min
5	Notifications (server function + schedule)	20 min

The app is **fully usable after Phase 4**. Phase 5 (notifications) is optional and can be done another day — tell him this so he doesn’t feel pressured.

2. PHASE 1 — Get the code into his own GitHub

Why: hosting reads the code from a repository, and this gives him his own copy he controls.

Step 1.1 — Create a GitHub account

Go to **github.com** -> Sign up. Free.

Step 1.2 — Create an empty repository

1. Click the + in the top right -> **New repository**.
2. **Repository name:** miturno_app
3. Visibility: **Private** is recommended (it's his personal app).
4. Do **not** check "Add a README file".
5. Click **Create repository**.

Step 1.3 — Upload the files (no command line needed)

On the empty repository page there is a link: **"uploading an existing file"**.

1. Click it.
2. Unzip the project folder Patricio gave him.
3. **Important:** drag the **contents** of the folder (index.html, the css folder, the js folder, sw.js, etc.) — **not** the outer folder itself. GitHub's uploader accepts folders when dragged.
4. Scroll down, click **Commit changes**.

Verify with him: the repository root must show index.html directly (not nested inside another folder). If he sees a single folder, he uploaded the wrapper — have him delete it and re-upload the contents.

Alternative if drag-and-drop misbehaves: he can skip GitHub entirely and deploy to Netlify by dragging the folder (see Phase 4, "Option B"). GitHub is recommended because it makes future updates easier, but it is not mandatory.

3. PHASE 2 — His own Supabase project

Why: this is his private database, for cloud backup and for scheduled notifications.

Step 2.1 — Create the project

1. Go to **supabase.com** -> Sign up (free tier — no credit card).
2. **New project**.
3. **Name:** mi-turno
4. **Database password:** click "Generate a password" and **save it in a password manager**. He will rarely need it, but losing it is a hassle. *(Never ask him to share it with you.)*
5. **Region:** the one closest to him.
6. Leave **Enable Data API** checked (the app requires it).
7. Click **Create new project** and wait ~2 minutes for provisioning.

Step 2.2 — Create the database tables

Go to **SQL Editor** -> **New query**, paste this, click **Run**:

```
-- Table that stores the app state (cloud sync)
create table if not exists public.app_state (
  user_id uuid primary key references auth.users(id) on delete cascade,
```

```

    data jsonb,
    updated_at timestampz default now()
);
alter table public.app_state enable row level security;
create policy "solo mis datos" on public.app_state
  for all to authenticated
  using (auth.uid() = user_id)
  with check (auth.uid() = user_id);

-- Table that stores the push notification subscription
create table if not exists public.push_subscriptions (
  user_id uuid primary key references auth.users(id) on delete cascade,
  subscription jsonb not null,
  prefs jsonb not null default '{}',
  tz text default 'America/Mexico_City',
  last_sent jsonb not null default '{}',
  updated_at timestampz default now()
);
alter table public.push_subscriptions enable row level security;
create policy "solo mis notif" on public.push_subscriptions
  for all to authenticated
  using (auth.uid() = user_id)
  with check (auth.uid() = user_id);

```

Expected result: **“Success. No rows returned.”**

Explain to him: those row level security lines are what guarantee that only he can read his own rows.

Note on his timezone: the default is America/Mexico_City. If he lives elsewhere, it doesn’t matter — the app writes his real timezone automatically when he enables notifications.

Step 2.3 — Turn off email confirmation

Go to **Authentication -> Sign In / Providers -> Email**, scroll down inside that panel, and **turn OFF “Confirm email”** (it may be labeled “Enable email confirmations”). Save.

Why: without this, signing up sends a confirmation link by email, which on iPhone opens in Safari instead of the installed app and creates confusion. It is his private app, so this is safe.

Step 2.4 — Copy his two public keys

Go to **Project Settings -> API** (gear icon). He needs:

1. **Project URL** — looks like `https://abcdefghijklk.supabase.co`
2. **Publishable key** (also shown as **anon public**) — a long string

These two are safe to paste into the chat and into the code — they are public by design and protected by the security rules from Step 2.2.

IMPORTANT: Tell him explicitly: he must **never** copy the `service_role` key. That one bypasses all security. If he ever pastes it somewhere public, he must rotate it immediately.

4. PHASE 3 — Put his keys into the app

Why: right now the code points at Patricio’s backend. This switches it to his own.

He edits **one file:** `js/config.js` — only three lines.

How to edit on GitHub (easiest, no tools)

1. In his repository, open the js folder -> click config.js.
2. Click the **pencil icon** (Edit this file).
3. Make the three changes below.
4. Scroll down -> **Commit changes**.

The three changes

Change 1 — line ~9, his name:

```
const DEFAULT_USER_NAME = "Patricio";
```

-> replace Patricio with his own first name.

Change 2 — lines ~19-20, his Supabase keys:

```
const SUPABASE_URL = "https://xeerkvjlguycmdrimfbn.supabase.co";  
const SUPABASE_KEY = "sb_publishable_lK_T9Z1R7-iHuykHk0N0ug_nvzaHYFG";
```

-> replace both values with **his** Project URL and **his** Publishable key from Step 2.4.

Change 3 — line ~23, his notification key:

```
const VAPID_PUBLIC = "BPKN-6oj8ac8FQcdqAb8LFzPKSXL4gqebi6k4IBVyFL8IUU326ffNY9BE0w0yhF1mDbpc1m";
```

-> replace with **his own public key**, generated as follows.

Generating his own notification keys

The project includes a generator that runs entirely in his browser — nothing is sent anywhere.

1. Have him open the file **generar-llaves.html** from the unzipped folder by double-clicking it (it opens in his browser).
2. Click **“Generar mis llaves”**.
3. Two values appear:
 - **Llave pública (VAPID_PUBLIC)** -> goes into config.js (Change 3) **and** later into Supabase as a secret.
 - **Llave privada (VAPID_PRIVATE)** -> goes **only** into Supabase secrets in Phase 5.
4. He must **save both** in his password manager before closing the page.

ChatGPT: do not ask him to send you the private key. If he offers, decline and tell him to keep it in his password manager.

If the generator page fails (very old browser), an alternative is any reputable “VAPID key generator” web tool, or running `npx web-push generate-vapid-keys` if he happens to have Node.js. The keys must be a P-256 pair in base64url format.

Important formatting warning

Tell him to change **only the text inside the quotation marks**. The quotes, the semicolons and the word `const` must stay exactly as they are. A single missing quote breaks the whole app (it will load as a blank screen).

5. PHASE 4 — Publish it and install it on the iPhone

Why: iPhone only allows installing a web app that is served over HTTPS. Netlify does that for free.

Option A — Netlify connected to GitHub (recommended)

1. Go to **netlify.com** -> Sign up (he can log in with GitHub).
2. **Add new site -> Import an existing project -> GitHub.**
3. Authorize Netlify, then select the `miturno_app` repository.
4. Build settings — **this matters**:
 - **Build command**: leave **empty**
 - **Publish directory**: `.` (a single dot) or leave as the root
 - There is no build step; it's a static site.
5. Click **Deploy**.
6. When it finishes he gets a URL like `https://random-name-123.netlify.app`.
7. Optional but nice: **Site configuration -> Change site name** -> something like `mi-turno-<hisname>`.

Bonus: with this option, any future change committed on GitHub redeploys automatically.

Option B — Netlify drag & drop (if GitHub gave him trouble)

Go to **app.netlify.com/drop** and drag the project folder there. Instant deploy, but future updates require dragging again.

Step 4.1 — Verify in a desktop browser first

Have him open the Netlify URL on his computer. He should see the app with the “Hoy” screen.

If he sees a blank white/black screen: it's almost certainly a typo in `config.js` (Phase 3). Have him open the browser console (F12 -> Console), send you the red error, and fix the quote/comma.

Step 4.2 — Install on the iPhone

1. On the iPhone, open the Netlify URL **in Safari** (must be Safari — not Chrome, not Brave, for the install step).
2. Tap the **Share** button (square with an arrow pointing up).
3. Scroll and tap **“Add to Home Screen” / “Agregar a pantalla de inicio”**.
4. Name it **Mi Turno** -> **Add**.
5. Close Safari and open the app from the **new icon**.

It should now run full screen, with no browser bar. **From this moment the app is fully usable.**

Step 4.3 — Quick sanity check

Ask him to confirm: - The bottom tab bar (Hoy - Progreso - Workouts - Metas - Ajustes) sits flush at the bottom edge. - He can tap a habit and it marks as done. - Going to **Ajustes** shows sections including **Nube** and **Notificaciones**.

Step 4.4 — Connect the cloud (his own account)

1. **Ajustes -> Nube.**
2. Enter his email and a password (min. 6 characters) -> **Crear cuenta.**
3. It should say connected. If he later installs on another device, he logs in with the same email/password and his data appears.

If it errors: see Section 9, “Cloud errors”.

6. PHASE 5 — Notifications (optional, do it last)

Why: a closed web app cannot wake itself up. A small function on his Supabase runs every 5 minutes and sends the notifications that are due.

There are four sub-steps. All happen in his Supabase dashboard.

Step 5.1 — Deploy the function

1. Supabase -> **Edge Functions** -> **Deploy a new function / Create function**.
2. **Name it exactly:** send-reminders
3. Open the file supabase/functions/send-reminders/index.ts from the project folder (any text editor, or view it on GitHub), **copy all of its contents**, and paste it into the Supabase editor, replacing whatever placeholder code is there.
4. Click **Deploy**.

Step 5.2 — Turn OFF “Verify JWT”

Go to the function’s **Settings** and make sure **“Verify JWT”** is **disabled**.

Why this matters: if it’s on, the scheduled job gets rejected with a 401 and notifications silently never arrive. This is the single most common failure point.

Step 5.3 — Add the three secrets

Supabase -> **Edge Functions** -> **Secrets** (in the left sidebar under MANAGE) -> add:

Name	Value
VAPID_PUBLIC	his public key (same one he put in config.js)
VAPID_PRIVATE	his private key (from the generator — never share it)
VAPID_SUBJECT	mailto: + his own email, e.g. mailto:hisname@gmail.com

Leave any pre-existing entries (SUPABASE_URL, SERVICE_ROLE_KEY, ...) untouched — those are provided automatically.

Step 5.4 — Schedule it every 5 minutes

Supabase -> **Cron** (sometimes under *Integrations* or *Database*) -> **Create job**:

- **Name:** mi-turno-reminders
- **Schedule:** */5 * * * *
- **Type: Supabase Edge Function**
 - If that option is greyed out, there’s a button **“Install pg_net extension”** — click it first, wait a few seconds, then the option becomes available.
- **Method:** POST
- **Edge Function:** send-reminders
- **Timeout:** change the default 1000 to **15000** (1 second is not enough for the function to run)
- **HTTP Headers / Body:** leave empty
- **Create**

Step 5.5 — Enable and test in the app

1. Open the app **from the Home Screen icon** (notifications do not work from a Safari tab).
2. **Ajustes -> Notificaciones**.
3. Set his three times (morning / afternoon / night) -> **Guardar horarios**.
4. Tap **Activar notificaciones** -> accept the iOS permission prompt.

5. Tap **Enviar notificación de prueba** -> a notification should arrive within seconds.

If the test works but scheduled ones don't: the cron job or "Verify JWT" is the problem (Steps 5.2 / 5.4).

Live test: have him set the afternoon time to ~7 minutes from now, save, and wait. It should arrive on its own.

7. Making the app his (first-run personalization)

The app ships with Patricio's seed data as an example. **Everything is editable inside the app — he never needs to touch code again.** Walk him through:

Ajustes

- **Perfil:** his name.
- **Identidades / Metas:** these are the identities he's building ("El que construye su cuerpo", etc.). He should delete the ones that don't apply and create his own — each with a name, color, icon, his personal "why", and motivational phrases.
- **Hábitos:** the daily habits, each optionally linked to an identity.
- **Compromisos:** things he wants to quit (they track a clean-day streak).
- **Comidas:** choose **Menú** (simple list of meals) or **Fichas** (exchange-portion system with categories and quotas). Only the chosen one shows up in "Hoy".
- **Datos y respaldo:** teach him to download a backup file now and then.

Workouts

- **Rutinas:** create routines by hand, or use **Importar JSON**. The repo includes RUTINAS-como-importar-json.md (format guide) and ejemplo-rutina.json (template).
- **Actividades:** Gym (strength, with routines) plus class-type activities like Kickboxing/Boxeo. He can add or remove these.
- **Unidad de peso:** kg or lb.

Metas

- Tapping a goal opens an immersive page: his "why", his phrases, stats, and a **vision board** where he can add photos and write what each one represents.
- **Photos stay on the device** (they are not uploaded to Supabase), so they don't consume cloud storage.

Hoy

- Sections can be **reordered**: press and hold a section header for about half a second, then drag it.
-

8. Things ChatGPT must remember while helping

The service-worker cache rule

The app caches itself for offline use. If he ever changes a file, the version number in sw.js (const CACHE = "mi-turno-vNN") **must be incremented**, otherwise the iPhone keeps serving the old version.

If he reports “I changed something but I don’t see it”: 1. Fully close the app (swipe it away in the app switcher) and reopen. 2. If still stale: remove it from the Home Screen and re-add it from Safari. That wipes the cache completely.

Free tier facts (2026)

- Supabase free tier: 500 MB database, 1 GB file storage, 2 projects. His usage will be a few MB at most — **it will stay free**.
- Supabase pauses free projects after **7 days of no activity**. Daily use plus the cron job keeps it awake. If it ever pauses, he just clicks “Resume” — no data is lost.
- Netlify free tier is more than enough for a personal app and does not pause.

iOS specifics

- Notifications require **iOS 16.4+** and the app **installed to the Home Screen**.
- The workout rest-timer alarm sounds while the app is open; iOS may mute audio when the app is fully backgrounded. Vibration and the flashing screen resume when he returns.
- The install step must be done in **Safari** (Chrome/Brave can’t add to Home Screen the same way).

What NOT to do

- Do not tell him to edit any file other than `js/config.js`.
 - Do not have him reuse Patricio’s Supabase URL, keys, or VAPID keys — that would defeat the entire privacy purpose.
 - Do not ask for, store, or display his private key, database password, or `service_role` key.
-

9. Troubleshooting (symptom -> cause -> fix)

App shows a blank screen after deploying

Cause: syntax error in `config.js` (a missing quote, comma, or semicolon). **Fix:** open the site on a computer, press F12 -> Console, read the red error. It names the line. Restore the punctuation. Remind him: only the text *inside* the quotes changes.

“Ajustes -> Nube” says it can’t read the cloud

Cause: the SQL from Step 2.2 wasn’t run, or ran with an error. **Fix:** re-run the SQL. Confirm in **Table Editor** that `app_state` and `push_subscriptions` exist.

Sign-up says the email needs confirmation

Cause: Step 2.3 wasn’t done. **Fix:** turn off “Confirm email” in Authentication -> Email, then sign in again.

“Invalid login credentials”

Cause: wrong password, or the account was created on a different Supabase project. **Fix:** verify the URL/key in `config.js` match the project he’s looking at. He can create the account again.

Test notification: “Edge Function returned a non-2xx status code”

Cause: the function isn’t deployed, or the secrets are missing. **Fix:** verify the function exists and is named exactly send-reminders; verify all three VAPID secrets exist and are spelled exactly. Check **Edge Functions -> send-reminders -> Logs** for the real error.

Test notification: “Failed to send a request to the Edge Function”

Cause: usually CORS or the function erroring at startup. **Fix:** confirm he pasted the **complete, current** index.ts (it contains a CORS block). Redeploy.

Test notification works, scheduled ones never arrive

Cause: (a) cron job not created, (b) “Verify JWT” still enabled, (c) timeout too low. **Fix:** Steps 5.2 and 5.4. Check the cron job’s run history in the Cron section — it shows whether each run succeeded.

No notification permission prompt appears on iPhone

Cause: he opened the app in Safari instead of from the Home Screen icon, or iOS < 16.4. **Fix:** install to Home Screen, open from the icon, try again. Verify iOS version in Settings -> General -> About.

Changes don’t appear on the phone

Cause: service-worker cache. **Fix:** see Section 8. Bump sw.js version, fully close and reopen; last resort, remove and re-add to Home Screen.

Bottom tab bar floats or leaves a gap

Cause: stale cached CSS from an older version. **Fix:** remove from Home Screen and re-add. The current code has this fixed.

Photos don’t save in the vision board

Cause: normally storage permissions/private browsing. **Fix:** make sure he’s using the installed app, not a private Safari tab. Photos live in IndexedDB on the device and are intentionally excluded from cloud sync and from backups.

10. Final checklist (go through this with him at the end)

- ☐ Code is in his own GitHub repository (or deployed by drag & drop)
 - ☐ config.js has **his** name, **his** Supabase URL, **his** Supabase publishable key, **his** VAPID public key
 - ☐ Both SQL tables created (app_state, push_subscriptions)
 - ☐ “Confirm email” turned OFF
 - ☐ Site deployed on Netlify with an HTTPS URL
 - ☐ App installed on the Home Screen and opens full screen
 - ☐ Signed up in **Ajustes -> Nube** and it says connected
 - ☐ (Optional) Function send-reminders deployed, Verify JWT off, three secrets set
 - ☐ (Optional) Cron job every 5 minutes, timeout 15000, pointing at the function
 - ☐ (Optional) Test notification received
 - ☐ He personalized his identities, habits, commitments and meals
 - ☐ He downloaded one backup file (Ajustes -> Datos y respaldo) and knows where it is
-

11. One-paragraph summary (if he asks “what am I actually doing?”)

“You’re taking an app that’s already built and putting your own copy online, with your own private database. GitHub stores the code, Netlify puts it on the internet with a web address, and Supabase is your private vault for backup and reminders. You don’t write any code — you only paste your own keys into one file so the app talks to *your* vault instead of your friend’s. Then you add it to your iPhone’s Home Screen and it behaves like a normal app.”

Guide written for ChatGPT to assist a non-technical user. The app itself requires no further development — everything is configurable from within its Settings screen.